SIKLUS PENGEMBANGAN SISTEM (*System Development Life Cycle-SDLC*)

PERTEMUAN 4

Analisis Sistem Dan Desain

Minarni@2022

Program Studi Teknik Informatika |Fakultas Teknik | Institut Teknologi Padang

Pengembangan Sistem

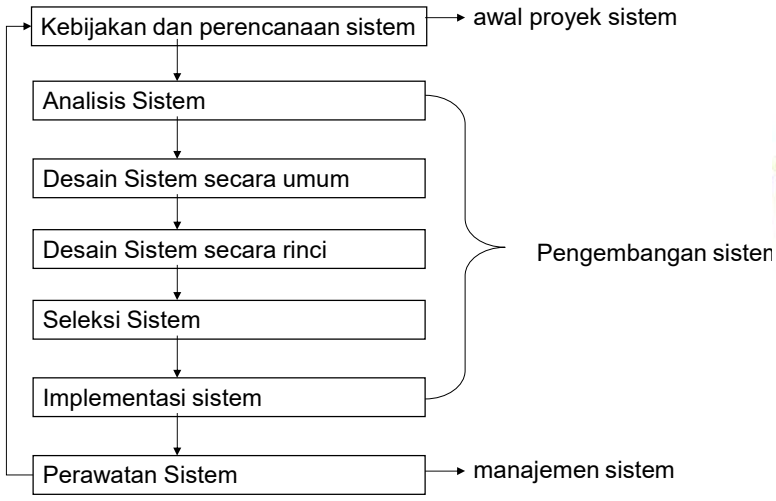
Pengembangan sistem adalah penyusunan suatu sistem yang baru untuk menggantikan sistem yang lama secara keseluruhan atau memperbaiki sistem yang telah ada.

Kenapa sistem perlu dikembangkan?

1. Adanya permasalahan yang timbul di sistem yang lama (ketidakberesan dan pertumbuhan organisasi)
2. Untuk meraih kesempatan
3. Adanya instruksi



Siklus Hidup Pengembangan Sistem (System Development Life Cycle)





Perangkat Lunak (software)

- Program komputer yang berhubungan dengan dokumentasi perangkat lunak seperti dokumentasi kebutuhan, model desain, cara penggunaan (user manual), dokumen teknis, maupun dokumen lainnya yang dapat mendukung program komputer untuk terus digunakan dan dikembangkan.

Sistem Perangkat Lunak

- Sistem yang memiliki komponen berupa perangkat lunak yang memiliki hubungan satu sama lain untuk memenuhi kebutuhan pelanggan (*customer*)

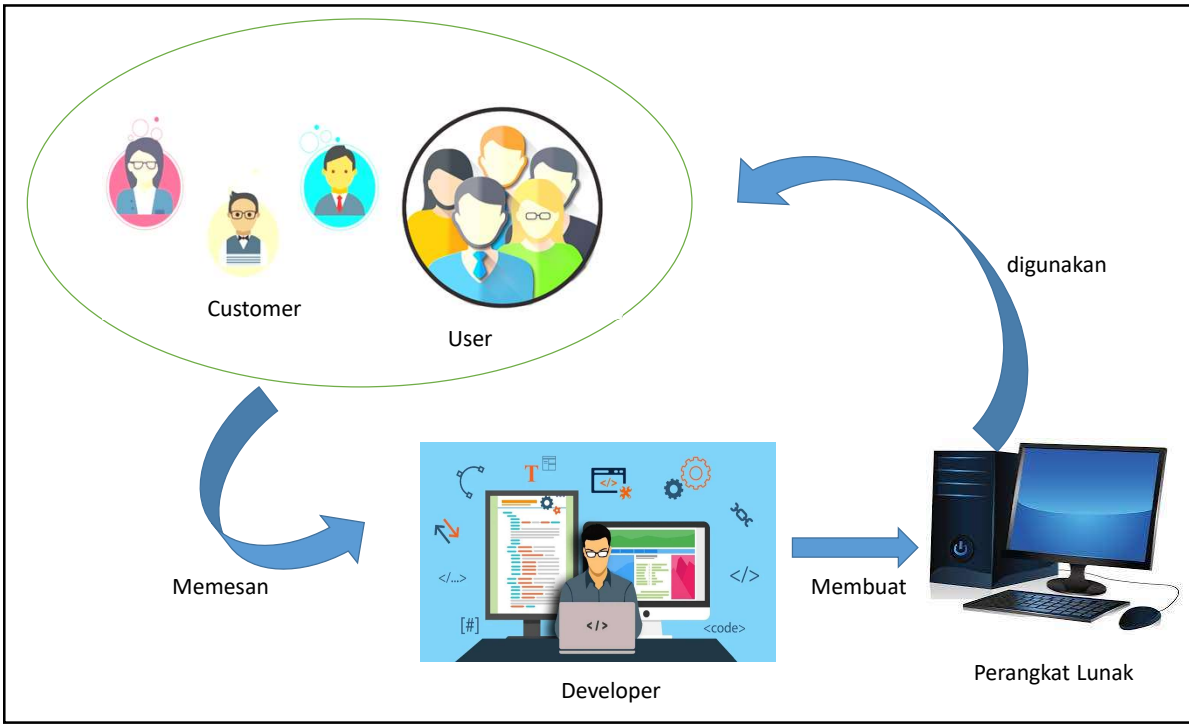
Rekayasa Perangkat Lunak (Software Engineering)

- Pembangunan dengan menggunakan prinsip atau konsep rekayasa dengan tujuan menghasilkan perangkat lunak yang bernilai ekonomi yang dipercaya dan bekerja secara efisien menggunakan mesin

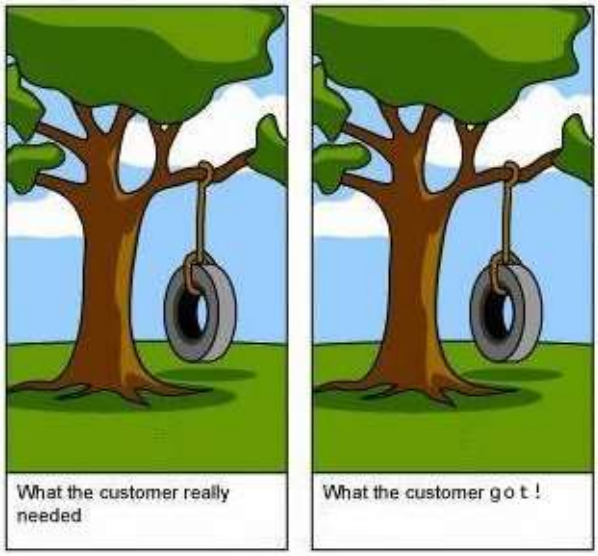
Keunikan dari Software

Karakteristik	Software	Hardware
Kompleksitas	Tingkat kompleksitas dari produk software tinggi, dengan kemungkinan perubahan parameter dan fungsi yang sangat beragam	Tingkat kompleksitas produk lain rendah, dengan kemungkinan perubahan parameter dan fungsi tidak beragam
Sumber Daya Manusia	Kuantitas SDM Tidak berhubungan dengan Kualitas dan Kecepatan Kerja	Kuantitas SDM berhubungan dengan Kualitas dan Kecepatan Kerja
Visibilitas Produk	Produk tidak terlihat dengan kasat mata, termasuk bila ada cacat (*defect*) dari produk	Produk terlihat dengan kasat mata, termasuk bila ada cacat (*defect*) dari produk

5

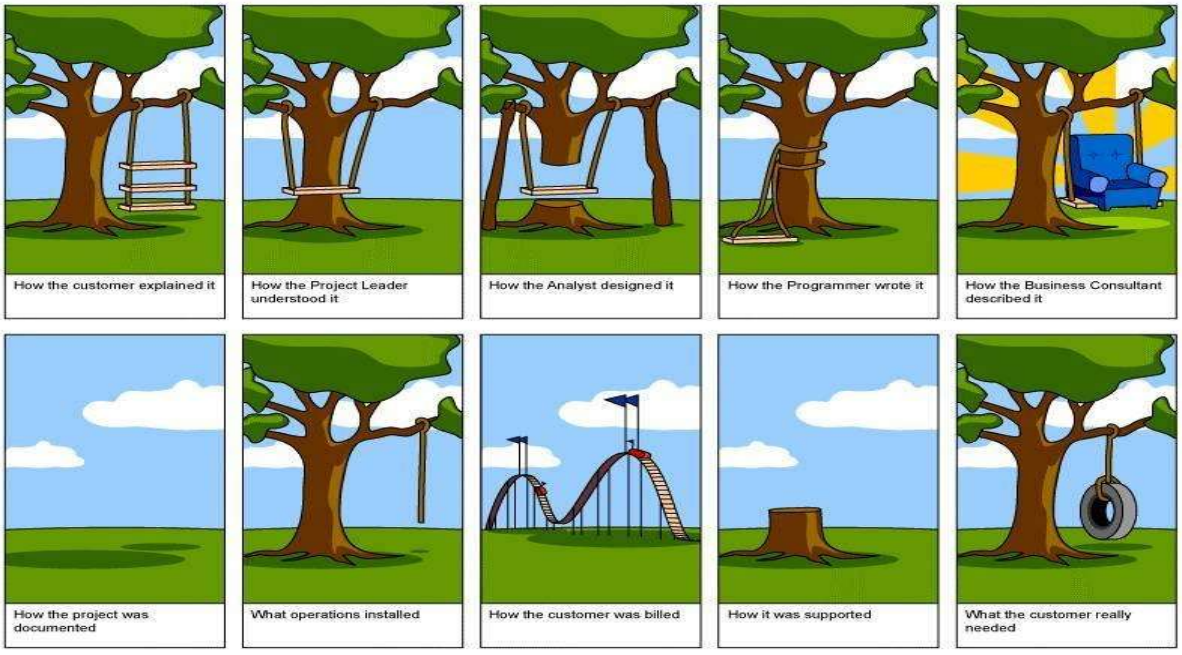


Tantangan Pengembangan Software



Sumber: Romi Satria Wahono/Systems Analysis and Design

7



Sumber: Romi Satria Wahono/Systems Analysis and Design

Siklus Pengembangan Software:

Alur, Peran, dan Tahapan (*Deliverable*)

1. **User/Product Owner** membawa permintaan kebutuhan (perubahan) software (System Request) ke System Analyst
2. **System Analyst** membuat analisis kelayakan (Feasibility Analysis) dari System Request tersebut
3. Setelah dinyatakan layak, System Analyst melakukan analysis dan design, dan hasilnya adalah System Specification
   - **Business Analyst** membantu System Analyst memahami proses bisnis dan knowledge berhubungan dengan software yang akan dibangun
4. System Specification diserahkan oleh System Anayst ke **Programmer** untuk dilakukan Konstruksi (*Coding*)
5. Hasil Konstruksi berupa Kode Program diserahkan ke **Software Tester** untuk diuji (*Unit, Integration, System, User Acceptance Testing*)
6. Kode Program + Dokumentasi (Pengembangan dan Penggunaa) + Laporan Testing = Software
7. Siklus kembali ke 1 apabila ada permintaan perubahan (Permintaan Perubahan Software)

Planning
(System Proposal)

Analysis and Design
(System Specification)

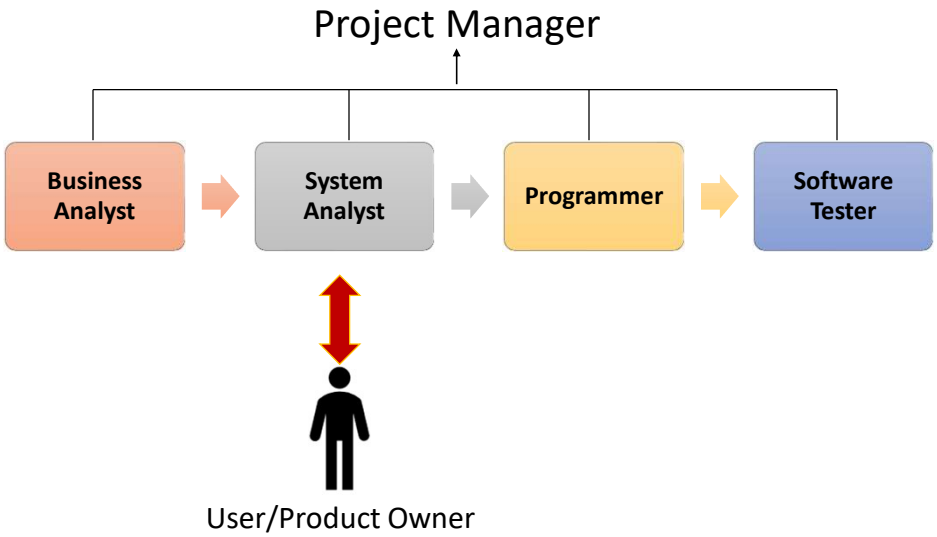
Implementation
(Software)

Maintenance
(Updated Software)

Sumber: Romi Satria Wahono/Systems Analysis and Design (Tilley, 2012) (Dennis, 2016) (Valacich, 2017)

9

Peran dalam Pengembangan Software



Sumber: Romi Satria Wahono/Systems Analysis and Design

10

User/Product Owner

- Pihak yang membutuhkan software (*requirement*), yang menganalisis bahwa ada business value (benefit) apabila suatu proses bisnis diautomasi menggunakan software
  - Kebutuhan dan business value ini diwujudkan dalam bentuk System Request
- Diperlukan skill, pengetahuan dan pemahaman tentang domain bidang dan proses bisnis
  - Bukan di bidang komputernya

Sumber: Romi Satria Wahono/Systems Analysis and Design

11

Business Analyst

- Fokus pekerjaan membantu user/product owner di masalah domain dan bidang (business) dari software
  - Nilai bisnis (business value) dari software (sistem)
  - Perbaikan bisnis proses yang diperlukan untuk pengembangan software
- Diperlukan skill, pengetahuan dan pemahaman tentang domain bidang dan proses bisnis
  - Bukan di bidang komputernya

Sumber: Romi Satria Wahono/Systems Analysis and Design

12

System Analyst

- Orang yang menganalisis kebutuhan bisnis, mengidentifikasi peluang untuk perbaikan, mendesain software (sistem) untuk mengimplementasikan idenya
- Fokus pekerjaan ke software dan sistem informasinya, bukan domain atau bidangnya
  - Bagaimana software bisa meningkatkan ROI (Return of Investment) dan mempercepat BEP (*Break Event Point*) dari layanan (organisasi)
  - Mendesain software dan sistem baru
  - Menjamin standard kualitas dari software dan sistem yang dibangun
- Diperlukan skill, pengetahuan dan pemahaman tentang design analysis, programming, business (*lesser degree*)

Sumber: Romi Satria Wahono/Systems Analysis and Design

13

Project Manager

- Tugas utama di perencanaan, schedule, budget dan monitoring pekerjaan
- Harus bisa memberi jaminan bahwa kebutuhan dan business value yang diharapkan user/product owner bisa terpenuhi
- Mampu mengelola member team development
  - Yang tidak semua memiliki kemampuan teknis dan komunikasi yang baik
- Apabila sumber daya manusia terbatas, atau project skala kecil, bisa dirangkap oleh Systems Analyst

Sumber: Romi Satria Wahono/Systems Analysis and Design

14

Systems Development Life Cycle (SDLC)

1. **Planning**: Mengapa Software harus Dikembangkan?
   - System request, feasibility analysis, project size estimation
2. **Analysis**: Siapa, Kapan Digunakan dan Alur Kerja Software?
   - Requirement gathering dan business process modeling
3. **Design**: Bagaimana Bekerja dan Komposisi dari Software?
   - Program design, user interface design, data design
4. **Implementation**: Konstruksi dan Penyerahan Software
   - System construction, testing, documentation, installation

Sumber: Romi Satria Wahono/Systems Analysis and Design

15

Planning

1. Identifikasi Business Value (System Request)
   1. Lower Costs
   2. Increase Productivity
   3. Increase Profits
2. Analisis Kelayakan
   1. Technical Feasibility
   2. Economic Feasibility
   3. Organizational Feasibility

(**System Proposal**)

16

Analysis

1. Pengumpulan dan analisis kebutuhan (Requirements):
   - Siapa yang menggunakan software?
   - Apa yang dilakukan oleh software?
   - Kapan software digunakan?
2. Investigasi Software yang Ada (Baseline)
3. Identifikasi Peluang untuk Perbaikan
4. Kembangan konsep untuk software baru

(**System Specification**)

17

Design

1. **Program Design**
   - Software seperti apa yang ingin dibuat
   - Komposisi dan arsitektur dari software
2. **User Interface Design**
   - Bagaimana pengguna berinteraksi dengan software
   - Pahami form/laporan yang digunakan oleh perusahaan
3. **Data Design**
   - Data apa yang akan disimpan
   - Format data yang disimpan
   - Dimana data akan disimpan

(**System Specification**)

18

Implementation

1. Konstruksi Software
   - Pembuatan kode program
2. Pengujian Software
   1. Unit Testing
   2. Integration Testing
   3. System Testing
   4. User Acceptance Test
3. Dokumentasi
   1. User Documentation
   2. System Documentation
4. Installation
   - Software lama dimatikan
   - Software baru diaktifkan (instalasi)

19

Metodologi Pengembangan Software (Model Process)

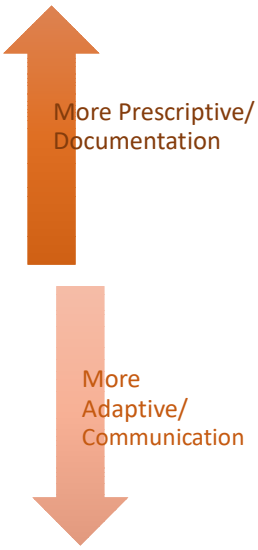
- A formalized approach to implementing the Software Development Life Cycle (SDLC) *(Dennis, 2012)*
- A simplified representation of a software process *(Sommerville, 2015)*
- A distinct set of activities, actions, tasks, milestones, and work products required to engineer high quality software *(Pressman, 2015)*

Sumber: Romi Satria Wahono/Systems Analysis and Design

20

Major Software Development Methodologies

1. **Structured Design**
   (Prescriptive) (1967-)
   - Waterfall method
   - Parallel development
2. **Rapid Application Development**
   (Iterative) (1985-)
   - Phased Development
   - Prototyping
3. **Agile Development**
   (Adaptive) (1995-)
   - Extreme Programming (XP)
   - Scrum

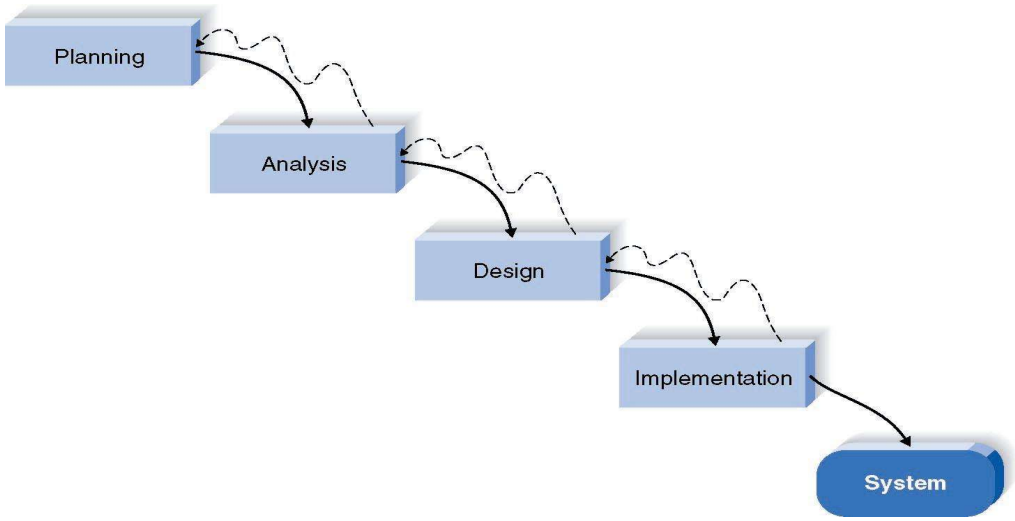


Sumber: Romi Satria Wahono/Systems Analysis and Design

Compiled from (Dennis, Wixom and Tegarden, 2016)

21

Waterfall Method

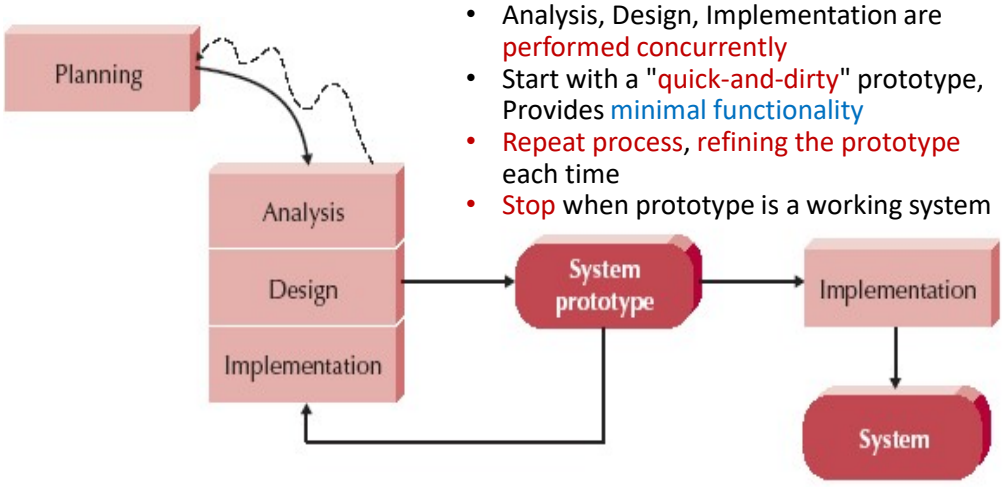


Sumber: Romi Satria Wahono/Systems Analysis and Design

22

Model Waterfall	• Model SDLC yang paling sederhana • Hanya cocok untuk pengembangan perangkat lunak dengan spesifikasi yang tidak berubah-ubah
Kelebihan	• Struktur pengembangan sistem jelas, tidak terjadi tumpang tindih pelaksanaan tahapan
Kekurangan	• Perubahan spesifikasi perangkat lunak terjadi ditengah alur pengembangan • Sangat sulit bagi pelanggan untuk mendefinisikan semua spesifikasi di awal tahap pengembangan • Pelanggan tidak mungkin bersabar mengakomodasi perubahan yang diperlukan di akhir tahap pengembangan

RAD: Prototyping



- Analysis, Design, Implementation are performed concurrently
- Start with a "quick-and-dirty" prototype, Provides minimal functionality
- Repeat process, refining the prototype each time
- Stop when prototype is a working system

Prototyping significantly reduces requirement and design errors, especially for user interfaces *(Boehm's First Law, Endres, 2013)* [***L3***]

Sumber: Romi Satria Wahono/Systems Analysis and Design

24

Model Prototype

- Termasuk dalam kategori pengembangan secara bertahap dan dapat digunakan untuk menyambungkan ketidakpahaman pelanggan
- Cocok digunakan untuk menggali spesifikasi kebutuhan pelanggan secara lebih detail tetapi beresiko tinggi terhadap membengkaknya biaya dan waktu proyek

Kelebihan

- Sangat cepat memberikan sistem bagi pengguna untuk berinteraksi (bahkan jika organisasi itu tidak siap/tidak memiliki gambaran)
- *Prototyping* meyakinkan klien bahwa tim proyek bekerja dengan baik (tidak ada penundaan yang lama di mana pengguna melihat kemajuan),
- *Prototyping* membantu lebih cepat memperbaiki persyaratan nyata (pengguna dapat berinteraksi dengan prototipe untuk lebih memahami apa yang bisa dan tidak bisa lakukan).

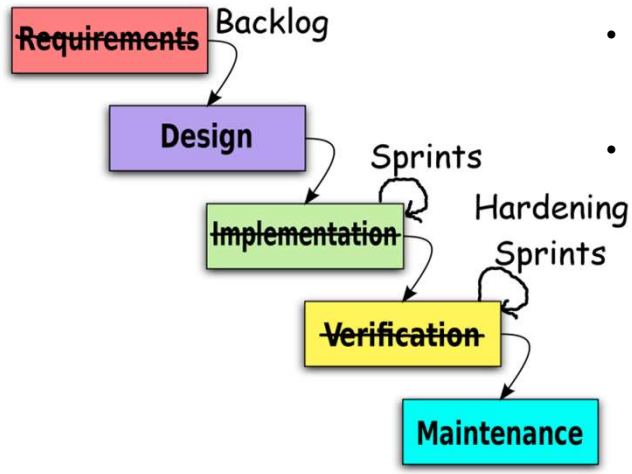
Kekurangan

- Sistem rilis yang cepat memiliki tantangan untuk mencoba melakukan dengan hati-hati pada fase analisis.
- Hal ini dapat menyebabkan masalah dalam pengembangan sistem yang kompleks karena isu dan permasalahan mendasar yang tidak diakui dengan baik sampai ke dalam proses pembangunan

Agile Development

- Menggunakan sedikit aturan yang mudah untuk dipelajari dan diikuti
- Mengurangi banyak pemodelan dan dokumentasi
- Menekankan kesederhanaan (simple) dan pengembangan aplikasi yang iteratif (berulang)
- Contoh pengembangan ini:
  - Extreme Programming (XP)
  - Scrum
  - Dynamic Systems Development Model (DSDM)

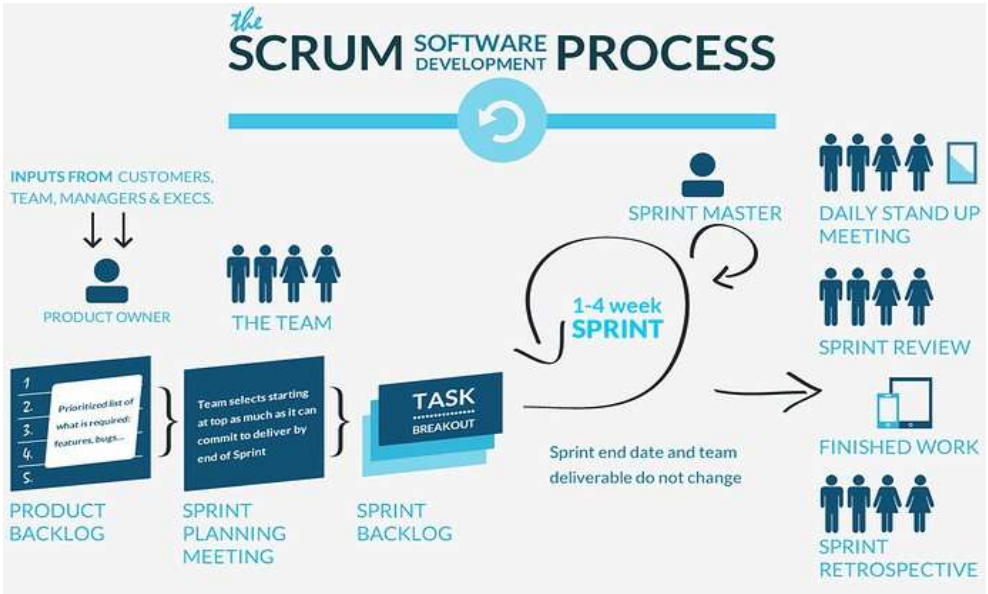
Scrum



- Tim terdiri dari 5-9 pengembang
- Tujuan dari Sprint adalah menentukan, memprioritaskan, dan memecah pekerjaan menjadi tugas yang lebih detail
- Tim memiliki karakter self-organized, bisa mengelola diri sendiri, memahami tugas, dan bertanggungjawab terhadap deadline
- Setiap Sprint menghasilkan fitur atau produk yang memiliki nilai dan menambahkan/memperbaiki fungsi sesuai kebutuhan user/product owner
  - perusahaan-perusahaan software seperti Microsoft, Adobe, Salesforce.com, Google, Netflix, Spotify

Sumber: Romi Satria Wahono/Systems Analysis and Design

29



Tahapan-tahapan dalam SCRUM:

1. Menentukan tim
Tahapan awal dalam memulai menerapkan metode Scrum adalah dengan menentukan anggota tim. Usahakan jumlah anggota seefektif dan seefisien mungkin.
2. Menentukan waktu pengerjaan
Sprint sendiri serangkaian pekerjaan yang dilakukan untuk menyelesaikan suatu masalah khususnya penciptaan produk baru. Setelah menemukan tim, langkah menjalankan metode Scrum selanjutnya adalah menentukan waktu pengerjaan atau *sprint.* Biasanya, *sprint* berlangsung 7 hingga 30 hari.
3. Menentukan peran dalam tim
Agar dapat bekerja maksimal, suatu tim haruslah memiliki struktur dan perannya masing-masing agar tidak tumpang tindih. Dalam *project management*, biasanya terdapat peran penting yang dimiliki oleh anggota tim.
Peran ini adalah Scrum *master,* atau menurut Solstice bisa juga disebut sebagai *project manager*.
Seorang *Scrum master* memastikan proyek berjalan dengan lancar dan sesuai. Peran selanjutnya dalam menjalankan metode Scrum adalah *product owner*, ia bertanggung jawab memastikan produk yang dihasilkan sesuai dengan kualitas yang diinginkan.

4. Mengumpulkan berbagai permasalahan
Langkah berikutnya adalah mengumpulkan berbagai hal yang didapat di lapangan. Dalam *project management*, hal ini disebut sebagai *backlog*. Berbagai permasalahan atau *backlog* ini kemudian dikumpulkan dan dibuat prioritas pengerjaannya.
5. Memulai *sprint*
Setelah semua langkah dijalankan, maka bisa langsung memulai *sprint* tersebut. Dalam melaksanakan *sprint*, bisa saja ditemukan permasalahan atau *backlog* lain. Komunikasikan hal ini dengan *product owner* apakah *backlog* tersebut bisa dilaksanakan dalam *sprint* tersebut atau *sprint* selanjutnya.

Selecting the Right Methodology

1. *Clarity of User Requirements* (Kejelasan Persyaratan Pengguna)
2. *Familiarity with Technology* (Kefamiliaran dengan teknologi)
3. *System Complexity* (Kompleksitas Sistem)
4. *System Reliability* (Keandalan Sistem)
5. Short Time Schedules (Jadwal Pendek)
6. Schedule Visibility

Strategi Pemilihan Metodologi - Matriks

Ability to Develop Systems	Structured Methodologies		RAD Methodologies			Agile Methodologies	
	Waterfall	Parallel	Phased	Prototyping	Throwaway Prototyping	XP	SCRUM
With Unclear User Requirements	Poor	Poor	Good	Excellent	Excellent	Excellent	Excellent
With Unfamiliar Technology	Poor	Poor	Good	Poor	Excellent	Good	Good
That Are Complex	Good	Good	Good	Poor	Excellent	Good	Good
That Are Reliable	Good	Good	Good	Poor	Excellent	Excellent	Excellent
With a Short Time Schedule	Poor	Good	Excellent	Excellent	Good	Excellent	Excellent
With Schedule Visibility	Poor	Poor	Excellent	Excellent	Good	Excellent	Excellent

34

Terimakasih